PLANO DE DESENVOLVIMENTO

gpdfview — reimplementação do qpdfview em Rust, GTK4 e libadwaita

Objetivo central
Construir um visualizador de PDF com paridade funcional com o qpdfview, preservando especialmente a rapidez de abertura, a navegação em documentos extensos e a eficiência de impressão.

Aplicação de referência	qpdfview (C++ / Qt)
Nova implementação	gpdfview (Rust / GTK4 / libadwaita)
Motor PDF proposto	Poppler GLib, com bindings Rust e acesso FFI complementar quando necessário
Renderização	Cairo / integração com GTK
Impressão	CUPS / IPP, priorizando envio direto do PDF quando suportado
Estimativa inicial	38–63 dias de desenvolvimento concentrado; revisar após a etapa de análise

Documento de planejamento técnico • Versão 1.0 • 02 de novembro de 2026

1. Visão geral e objetivos

O gpdfview será uma reimplementação do qpdfview, não uma conversão literal de código C++ para Rust. A meta é preservar a experiência, os recursos e os resultados observáveis do programa original, substituindo a camada Qt por GTK4/libadwaita e adotando uma arquitetura modular em Rust.

Objetivos obrigatórios

- Reproduzir os principais fluxos de uso, menus, atalhos, abas, painéis e configurações do qpdfview.
- Abrir documentos grandes sem aguardar a renderização de todas as páginas.
- Exibir a primeira página rapidamente e manter navegação responsiva.
- Imprimir com fidelidade, evitando rasterização integral sempre que o caminho de impressão aceitar o PDF original.
- Controlar consumo de RAM, CPU e tarefas simultâneas.
- Eliminar dependência de Qt no produto final, sem impedir o uso de bibliotecas nativas especializadas.
- Validar paridade funcional e desempenho por testes comparativos reproduzíveis.

Princípio arquitetural

Não desenvolver um motor PDF próprio. Usar Poppler como mecanismo de interpretação e renderização, mantendo em Rust a lógica da aplicação, o ciclo de vida dos documentos, a navegação, o cache, as tarefas assíncronas e a interface.

2. Arquitetura técnica recomendada

CAMADA	RESPONSABILIDADES	TECNOLOGIAS
Interface	Janela, abas, menus, diálogos, atalhos, painel lateral e área de visualização.	GTK4, libadwaita, GActions, GIO
Aplicação	Documentos, sessões, estado por aba, navegação, preferências e coordenação de tarefas.	Rust
Motor PDF	Leitura, metadados, páginas, texto, sumário, permissões e renderização.	Poppler GLib + bindings Rust/FFI
Desenho	Superfícies, composição visual e saída de impressão renderizada.	Cairo, integração GTK
Impressão	Diálogo, opções, fila, envio direto de PDF e alternativa renderizada.	CUPS / IPP / GTK Print
Persistência	Preferências, histórico, favoritos e restauração de sessão.	GSettings e/ou SQLite

Nota de integração: confirmar a versão de Poppler disponível nas distribuições-alvo e a cobertura real dos crates escolhidos. Não presumir que um binding de alto nível exponha toda a API necessária.

3. Estratégia de desempenho

A rapidez do qpdfview na abertura e impressão de PDFs grandes é o principal requisito de qualidade do gpdfview. O desenho deve privilegiar trabalho incremental e sob demanda.

Abertura e primeira página

- Abrir o documento e obter apenas os metadados necessários para iniciar a visualização.
- Renderizar primeiro a página atual; não percorrer nem converter o documento inteiro na abertura.
- Mostrar a primeira página assim que o motor disponibilizar uma superfície válida.
- Evitar operações síncronas demoradas na thread principal do GTK.

Renderização e cache

- Executar renderização em workers, com limite de concorrência configurável.
- Priorizar a página visível; depois, páginas adjacentes e miniaturas necessárias.
- Implementar cache LRU com limite por bytes, não apenas por quantidade de páginas.
- Separar cache de páginas em resolução de leitura e cache de miniaturas.
- Cancelar ou descartar resultados de tarefas que se tornaram obsoletas após navegação ou mudança de zoom.
- Recalcular dimensões de renderização conforme zoom, DPI e escala da tela.
- Liberar superfícies e referências nativas de forma previsível.

Concorrência e segurança

- Manter objetos GTK e atualizações de interface na thread principal.
- Definir claramente quais objetos Poppler podem ser usados em workers; quando necessário, usar instâncias/documentos separados ou serializar acesso.
- Evitar compartilhar ponteiros GObject sem regras explícitas de propriedade e thread-safety.
- Usar canais e mensagens para entregar resultados dos workers à interface.
- Instrumentar filas, cancelamentos, tempo de renderização e tamanho do cache.

Regra de ouro: a navegação não pode ficar bloqueada esperando miniaturas, pesquisa de texto, pré-renderização ou leitura de páginas que não estão visíveis.

4. Impressão: requisito de primeira classe

A impressão deve ser projetada e testada desde o início, não deixada para o fim. A melhor rota depende das capacidades da fila de impressão e do backend disponível.

CAMINHO	COMPORTAMENTO	CUIDADOS
A — PDF direto	Enviar o PDF original à infraestrutura de impressão quando a fila/backend aceitar esse formato.	Preservar vetores e fontes; validar opções, intervalos, orientação e compatibilidade real da fila.
B — Renderizado	Renderizar cada página necessária com Poppler/Cairo e encaminhar a saída ao sistema de impressão.	Processar página por página; controlar resolução e memória; não rasterizar o documento inteiro de uma vez.

- Comparar impressão do gpdfview e qpdfview com a mesma impressora, driver, arquivo e opções.
- Verificar PDFs com transparência, fontes incorporadas, imagens de alta resolução, páginas em tamanhos mistos e documentos protegidos.
- Registrar tempo até o envio, tempo total, tamanho do spool e pico de memória.
- Não afirmar que o envio direto será sempre possível: a compatibilidade depende da fila, do backend e do caminho de impressão adotado.

5. Matriz de paridade funcional

A implementação deve ser guiada por uma matriz de rastreabilidade: cada recurso do qpdfview recebe um requisito, um módulo responsável, um teste e um estado de conclusão.

PRIORIDADE	FUNCIONALIDADES	CRITÉRIO DE CONCLUSÃO
P0 — essencial	Abrir/fechar PDF; múltiplos documentos; abas; navegação; zoom; ajuste à largura/página; layout contínuo; rotação; pesquisa; miniaturas; sumário; impressão; sessão básica.	Fluxos principais funcionam e são cobertos por testes de regressão.
P1 — paridade avançada	Tela cheia; apresentação; propriedades; histórico; favoritos; configurações por documento; atalhos configuráveis; anotações; formulários.	Comportamento documentado e compatibilidade validada com amostras reais.
P2 — extensões	SyncTeX; suporte a DjVu/PostScript por módulos ou backends adicionais; integrações específicas.	Implementado somente após estabilização do núcleo PDF.

Atenção: anotações, formulários, assinaturas, recursos multimídia e recursos específicos de formatos podem depender de capacidades e limitações da versão do Poppler. Fazer uma prova técnica antes de assumir paridade total.

6. Estrutura sugerida do repositório

```
gpdfview/
├── Cargo.toml
├── src/
│ ├── main.rs / application.rs / window.rs
│ ├── document/ # ciclo de vida, metadados, sessão
│ ├── engine/ # Poppler, renderização, texto, sumário
│ ├── viewer/ # canvas, layout, zoom, navegação, cache
│ ├── printing/ # CUPS/IPP, jobs e opções
│ ├── ui/ # headerbar, toolbar, abas, sidebar, diálogos
│ ├── config/ # preferências e persistência
│ └── utils/ # atalhos, logs e instrumentação
├── data/ # ícones, recursos e arquivo .desktop
├── tests/ # documentos, desempenho e impressão
└── README.md
```

Separar interface, estado da aplicação e motor PDF. A interface não deve chamar Poppler diretamente em diversos pontos do código; deve consumir serviços bem definidos do módulo engine.

7. Fases e cronograma estimado

FASE	ENTREGAS PRINCIPAIS	ESTIMATIVA
1. Engenharia reversa	Mapear classes, fluxos, cache, impressão, atalhos, menus e dependências do qpdfview.	3–5 dias
2. Fundação	Projeto Cargo, GTK4/libadwaita, integração Poppler/Cairo, janela e abertura de arquivos.	4–7 dias
3. Núcleo PDF	Documento, metadados, páginas, navegação, zoom, senha e renderização básica.	5–8 dias
4. Desempenho	Workers, cache LRU, pré-renderização, cancelamento e testes com PDFs extensos.	5–8 dias
5. Leitura	Pesquisa, miniaturas, sumário, layouts, rotação e tela cheia.	5–8 dias
6. Impressão	PDF direto, alternativa renderizada, diálogo, opções e comparação de resultados.	5–8 dias
7. Paridade	Configurações, favoritos, histórico, anotações, formulários e atalhos.	7–12 dias
8. Estabilização	Testes, correções, integração desktop, empacotamento e documentação.	4–7 dias

Total indicativo: 38–63 dias. A estimativa pressupõe desenvolvimento concentrado por uma pessoa experiente. A etapa de engenharia reversa pode alterar significativamente o prazo; tarefas de integração, depuração e compatibilidade com impressoras são fontes relevantes de incerteza.

8. Plano de testes e critérios de aceitação

Construir uma coleção fixa de PDFs de teste e executar qpdfview e gpdfview no mesmo ambiente. Registrar versões de kernel, Poppler, GTK, driver de impressão, CPU, RAM e armazenamento para que os resultados sejam comparáveis.

AMOSTRA	OBJETIVO	MÉTRICAS
PDF pequeno (10 páginas)	Validar fluxo básico e latência de abertura.	Tempo até janela e primeira página.
PDF longo (500–5.000 páginas)	Validar abertura sem varredura integral e navegação.	Primeira página, troca de página, RAM e CPU.
PDF grande (centenas de MB)	Validar I/O, memória e estabilidade.	Tempo de abertura, pico de RAM, erros.
PDF com imagens/fontes complexas	Validar fidelidade e renderização.	Tempo por página e comparação visual.
PDF protegido	Validar senha e permissões.	Abertura, autenticação e comportamento.
PDF de impressão extensa	Validar spool e saída física.	Tempo, RAM, tamanho do spool e fidelidade.

Critérios de aceitação recomendados

- A primeira página aparece sem esperar pela geração de miniaturas ou pela renderização do documento inteiro.
- A interface permanece responsiva durante renderização, pesquisa e impressão.
- O uso de memória cresce de forma controlada e respeita o limite configurado do cache.
- A navegação rápida não acumula tarefas de renderização inúteis.
- A impressão mantém conteúdo, escala, orientação, fontes e qualidade equivalentes ao resultado de referência.
- Nenhuma regressão crítica é aceita sem justificativa e decisão explícita.

Não fixar metas absolutas de milissegundos antes de medir a máquina de referência. Primeiro medir qpdfview; depois definir limites de regressão realistas para o gpdfview.

9. Recomendações e riscos técnicos

Recomendações prioritárias

- Fazer engenharia reversa antes de codificar: estudar o código-fonte e observar o comportamento real do qpdfview, especialmente o fluxo de impressão e o cache.
- Provar Poppler cedo: criar um protótipo mínimo para abrir, renderizar, pesquisar e imprimir antes de definir abstrações definitivas.
- Validar threads e ownership: documentar as regras de vida útil dos objetos GObject e das superfícies Cairo.
- Medir desde o primeiro commit: adicionar logs estruturados e benchmarks automatizados para abertura, renderização, cache e impressão.
- Manter testes de regressão visual: comparar páginas renderizadas com tolerância definida para diferenças de antialiasing.
- Separar recursos por capacidade: identificar o que Poppler oferece diretamente, o que exige FFI e o que requer implementação adicional.

- Empacotar de forma reproduzível: testar em BigLinux e em pelo menos outra distribuição GTK, documentando versões mínimas de bibliotecas.
- Revisar licenças: verificar GPL do qpdfview, Poppler e dependências, avisos de copyright, distribuição de binários e obrigações de código-fonte.

Riscos que exigem validação

RISCO	IMPACTO	MITIGAÇÃO
Bindings incompletos	Bloqueio de recursos ou necessidade de FFI.	Prova de conceito e bindings específicos para APIs ausentes.
Thread-safety do motor	Travamentos, corrupção ou lentidão.	Definir modelo de workers e propriedade de objetos; testar stress.
Impressão não aceita PDF direto	Perda de velocidade ou fidelidade.	Detectar capacidade do backend e manter fallback por página.
Diferenças de comportamento	Paridade incompleta com qpdfview.	Matriz de equivalência, testes funcionais e documentação de desvios.
Memória em documentos extremos	OOM ou degradação de navegação.	Cache limitado por bytes, cancelamento e testes com arquivos extremos.

10. Primeiro marco: protótipo de desempenho

O primeiro marco deve ser deliberadamente pequeno. Antes de reproduzir toda a interface, comprovar que a base técnica consegue abrir e imprimir documentos grandes com desempenho competitivo.

Entregáveis do marco

- Aplicação GTK4/libadwaita que abre um PDF por seletor de arquivos.
- Integração funcional com Poppler GLib.
- Exibição da primeira página e navegação por número de página.
- Zoom básico e ajuste à largura.
- Renderização em worker, sem bloquear a interface.
- Cache limitado por memória e pré-renderização de páginas adjacentes.
- Teste de impressão com caminho direto quando suportado e fallback renderizado.
- Relatório comparativo de desempenho com qpdfview.

Definição de pronto (DoD)

O marco só será considerado concluído quando os testes com arquivos de referência forem repetíveis, não houver bloqueio perceptível da interface durante renderização e a impressão tiver sido validada em pelo menos um backend real.

Sequência recomendada de execução

1	Obter o código-fonte e a versão exata do qpdfview usada como referência.
2	Mapear classes e fluxos de abertura, renderização, cache e impressão.
3	Inventariar APIs Poppler utilizadas e confirmar equivalentes no Poppler GLib.
4	Criar protótipo Rust mínimo e validar integração com GTK4, Poppler e Cairo.
5	Medir qpdfview com o conjunto de PDFs de referência.
6	Implementar abertura, primeira página, navegação e impressão no gpdfview.
7	Comparar resultados e ajustar a arquitetura antes de expandir funcionalidades.

Conclusão

O sucesso do gpdfview depende menos de trocar C++ por Rust e mais de preservar as decisões de engenharia que tornam o qpdfview eficiente. A prioridade deve ser validar Poppler, concorrência, cache e impressão com documentos reais. Com esse núcleo comprovado, a reprodução da interface e a expansão da paridade funcional tornam-se etapas controláveis.

Diretriz final: primeiro equivalência de desempenho no núcleo; depois paridade funcional ampla; por último, refinamento visual e distribuição.

Referências técnicas para consulta

• qpdfview: https://github.com/lumina-desktop/qpdfview (confirmar o repositório e a origem exata da versão de referência antes do levantamento).
• Poppler: https://poppler.freedesktop.org/
• GTK4 Rust: https://gtk-rs.org/gtk4-rs/stable/latest/docs/gtk4/
• libadwaita Rust: https://gtk-rs.org/gtk4-rs/stable/latest/docs/libadwaita/
• CUPS: https://openprinting.github.io/cups/
• Cairo: https://www.cairographics.org/